Name	**Yedoti Premchand**
Roll Number	**23BTRCT074**
HCL ID	**HCLTFP2260962**
College Name	**JAIN University**
Batch	**2027**
Date	**28/10/2026**
Title	**HCLTECH – CLOUD & DATA ENGINEERING ASSESSMENT 2**

SECTION A: ADVANCED SQL

Question 1: Database Design and Advanced Queries

Created a database named EnterpriseDB

Untitled-1.sql ●

```
1   •CREATE DATABASE EnterpriseDB;
2
3   •USE EnterpriseDB;
4
```

Created the tables Employees and Departments

```
6   •CREATE TABLE Departments (
7       DepartmentID INT PRIMARY KEY,
8       DepartmentName VARCHAR(100) NOT NULL,
9       Location VARCHAR(100));
10
11
12  •CREATE TABLE Employees (
13      EmployeeID INT PRIMARY KEY,
14      EmployeeName VARCHAR(100) NOT NULL,
15      DepartmentID INT,
16      Salary DECIMAL(10,2) NOT NULL,
17      JoiningDate DATE,
18      ManagerID INT,
19          FOREIGN KEY (DepartmentID)
20          REFERENCES Departments(DepartmentID),
21           FOREIGN KEY (ManagerID)
22          REFERENCES Employees(EmployeeID));
23
```

Inserted employee records and department record

```sql
INSERT INTO Departments VALUES
(1, 'IT', 'Bengaluru'),
(2, 'HR', 'Chennai'),
(3, 'Finance', 'Mumbai'),
(4, 'Sales', 'Hyderabad'),
(5, 'Marketing', 'Pune'),
(6, 'IT', 'Bengaluru');

INSERT INTO Employees VALUES
(101, 'Prem', 1, 95000, '2023-01-10', 101),
(102, 'Priya', 2, 80000, '2022-05-15', NULL),
(103, 'Nandu', 3, 85000, '2021-03-20', 102),
(104, 'Mukund', 4, 75000, '2023-07-01', NULL),
(105, 'Kiran', 1, 70000, '2024-02-10', 101),
(106, 'Divya', 1, 65000, '2024-04-12', 101),
(107, 'Manoj', 1, 55000, '2025-01-15', 101),
(108, 'Anjali', 2, 60000, '2025-02-20', 102),
(109, 'Vijay', 2, 45000, '2025-03-05', NULL),
(110, 'Meena', 3, 72000, '2024-08-18', 103),
(111, 'Ravi', 3, 50000, '2025-05-12', 103),
(112, 'Pooja', 4, 65000, '2024-09-25', 104),
(113, 'Suresh', 4, 40000, '2025-06-30', 104),
(114, 'Neha', 1, 65000, '2025-08-11', 101),
(115, 'Sai', 3, 55000, '2025-09-17', 103),
(116, 'Chandu', 1, 65000,  '2023-01-10', NULL);
```

Display employee names, department names, and manager names using joins.

```sql
SELECT
    e.EmployeeName,
    d.DepartmentName,
    m.EmployeeName AS ManagerName
    FROM Employees e
    LEFT JOIN Departments d
    ON e.DepartmentID = d.DepartmentID
    LEFT JOIN Employees m
    ON e.ManagerID = m.EmployeeID;
```

Output | Result #8

EmployeeName	DepartmentName	ManagerName
Prem	IT	Prem
Priya	HR	NULL
Nandu	Finance	Priya
Mukund	Sales	NULL
Kiran	IT	Prem
Divya	IT	Prem
Manoj	IT	Prem
Anjali	HR	Priya
Vijay	HR	NULL
Meena	Finance	Nandu
Ravi	Finance	Nandu
Pooja	Sales	Mukund
Suresh	Sales	Mukund
Neha	IT	Prem
Sai	Finance	Nandu
Chandu	IT	NULL

Employees earning more than their department's average salary

```
61
62  SELECT EmployeeName, DepartmentID, Salary
63  FROM Employees e
64  WHERE Salary > (
65      SELECT AVG(Salary)
66      FROM Employees
67      WHERE DepartmentID = e.DepartmentID
68  );
```

Output | Result #9

EmployeeName	DepartmentID	Salary
Prem	1	95000.00
Priya	2	80000.00
Nandu	3	85000.00
Mukund	4	75000.00
Kiran	1	70000.00
Meena	3	72000.00
Pooja	4	65000.00

The second-highest distinct salary without using LIMIT, TOP, or FETCH FIRST.

```
70  SELECT MAX(Salary) AS SecondHighestSalary
71  FROM Employees
72  WHERE Salary < (
73      SELECT MAX(Salary)
74      FROM Employees);
75
```

Output | Result #10 | Result #11

SecondHighestSalary
85000.00

Rank employees by salary within each department and display the top three employees per department, retaining ties at the cutoff

```
75
76  SELECT *
77  FROM (
78      SELECT EmployeeName, DepartmentID, Salary,
79      RANK() OVER (
80      PARTITION BY DepartmentID
81      ORDER BY Salary DESC
82      ) AS SalaryRank
83      FROM Employees
84    ) AS Ranked
85  WHERE SalaryRank <= 3
86  ORDER BY DepartmentID, Salary DESC;
```

Output | Result #11

EmployeeName	DepartmentID	Salary	SalaryRank
Prem	1	95000.00	1
Kiran	1	70000.00	2
Divya	1	65000.00	3
Neha	1	65000.00	3
Chandu	1	65000.00	3
Priya	2	80000.00	1
Anjali	2	60000.00	2
Vijay	2	45000.00	3
Nandu	3	85000.00	1
Meena	3	72000.00	2
Sai	3	55000.00	3
Mukund	4	75000.00	1
Pooja	4	65000.00	2
Suresh	4	40000.00	3

Question 2: Sales Analytics and Query Optimization

Created a Sales table

```
91  CREATE TABLE Sales (
92      SaleID INT PRIMARY KEY,
93      CustomerID INT,
94       SaleDate DATE,
95       ProductName VARCHAR(100),
96       Quantity INT,
97       UnitPrice DECIMAL(10,2) );
```

Inserted Sales Records

```
99    INSERT INTO Sales VALUES
100   (1, 101, '2025-01-10', 'Laptop', 2, 50000),
101   (2, 102, '2025-01-15', 'Mouse', 5, 500),
102   (3, 103, '2025-01-20', 'Keyboard', 3, 1500),
103   (4, 101, '2025-02-05', 'Laptop', 1, 50000),
104   (5, 104, '2025-02-12', 'Mouse', 10, 500),
105   (6, 102, '2025-02-18', 'Monitor', 2, 12000),
106   (7, 105, '2025-03-02', 'Laptop', 2, 50000),
107   (8, 103, '2025-03-10', 'Keyboard', 4, 1500),
108   (9, 104, '2025-03-15', 'Monitor', 1, 12000),
109   (10, 101, '2025-04-05', 'Mouse', 8, 500),
110   (11, 105, '2025-04-12', 'Laptop', 1, 50000),
111   (12, 102, '2025-04-20', 'Monitor', 3, 12000),
112   (13, 103, '2025-05-05', 'Keyboard', 2, 1500),
113   (14, 104, '2025-05-15', 'Laptop', 1, 50000),
114   (15, 105, '2025-05-25', 'Mouse', 6, 500),
115   (16, 102, '2025-02-18', 'Monitor', 2, 12000);
116
```

Total revenue for each product

```
118
119   SELECT
120   ProductName,
121   SUM(Quantity * UnitPrice) AS TotalRevenue
122   FROM Sales
123   GROUP BY ProductName;
124
```

Output | Result #15 | Result #16

ProductName	TotalRevenue
Laptop	350000.00
Mouse	14500.00
Keyboard	13500.00
Monitor	96000.00

Monthly revenue and month-over-month revenue

```
125   SELECT
126       MONTH(SaleDate) AS SaleMonth,
127       SUM(Quantity * UnitPrice) AS MonthlyRevenue,
128       LAG(SUM(Quantity * UnitPrice)) OVER (
129       ORDER BY MONTH(SaleDate)
130       ) AS PreviousMonthRevenue
131   FROM Sales
132   GROUP BY MONTH(SaleDate)
133   ORDER BY SaleMonth;
134
```

Output | Result #16

SaleMonth	MonthlyRevenue	PreviousMonthRevenue
1	107000.00	NULL
2	103000.00	107000.00
3	118000.00	103000.00
4	90000.00	118000.00
5	56000.00	90000.00

To improve the potential for index usage
SELECT * FROM Employees WHERE YEAR(JoiningDate) = 2025;

```
135  SELECT *
136  FROM Employees
137  WHERE JoiningDate >= '2025-01-01'
138    AND JoiningDate < '2026-01-01';
```

Output | Result #17

EmployeeID	EmployeeName	DepartmentID	Salary	JoiningDate	ManagerID
107	Manoj	1	55000.00	15/01/2025	101
108	Anjali	2	60000.00	20/02/2025	102
109	Vijay	2	45000.00	05/03/2025	NULL
111	Ravi	3	50000.00	12/05/2025	103
113	Suresh	4	40000.00	30/06/2025	104
114	Neha	1	65000.00	11/08/2025	101
115	Sai	3	55000.00	17/09/2025	103

- This query retrieves employees who joined in 2025.
- It avoids applying the YEAR() function to JoiningDate.
- If an index exists on JoiningDate, the database can potentially use it to find records faster.

Question 3: SQL Troubleshooting

execution plan helps identify expensive operations in a query.
An execution plan shows how SQL runs a query. It helps us find what is making the query slow, such as reading the entire table or performing costly operations.

```
SELECT *
FROM Employees
WHERE DepartmentID = 1;
```

an appropriate index strategy for queries filtering employees by DepartmentID and Salary.

```
CREATE INDEX idx_dept_salary
ON Employees (DepartmentID, Salary);
```

This index helps the database to find the employees by department and salary faster, especially when the table contains millions of records.

query to identify duplicate customer email addresses and display only addresses occurring more than once.

```
SELECT Email, COUNT(*) AS Total
FROM Customers
GROUP BY Email
HAVING COUNT(*) > 1;
```

SECTION B: LINUX

Question 4: Log Analysis

To find all lines containing ERROR
-- grep "ERROR" /var/log/application.log
By using the key word " grep " we can display the all lines

To display the 20 most recent error entries
grep "ERROR" /var/log/application.log | tail -n 20

Count the total number of error entries
grep -c "ERROR" /var/log/application.log
-c counts the lines containing the ERROR

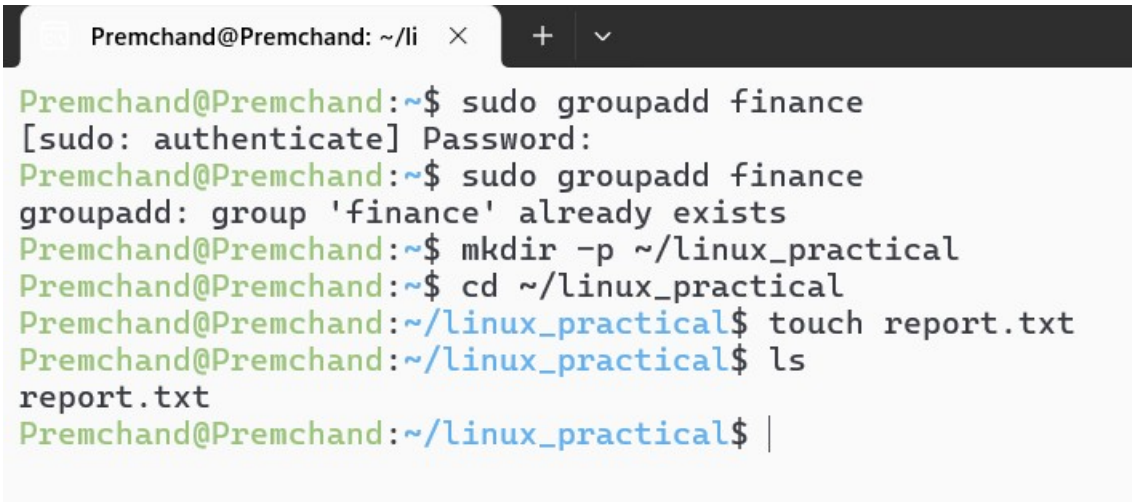
Extract error entries for a specified date into a separate file
grep "2026-10-28 ERROR" /var/log/application.log > errors_2026-10-28.log
Finds the error entries for the specified date and saves them into errors_2026-10-28.log

Calculate the number of errors occurring in each hour.

grep "ERROR" selects error lines, **sort** groups matching hours together, and uniq -c counts errors for each hour.

Question 5: Permissions and System Troubleshooting

Created a group named finance and the report file



Set permissions

We need these permissions:

Owner: read and write

Finance group: read only

Other users: no access

To identify the processes consuming the most CPU

```
Premchand@Premchand:~/linux_practical$ top
top - 06:23:01 up 7 min,  1 user,  load average: 0.00, 0.00, 0.00
Tasks:  26 total,   1 running,  25 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   5927.7 total,   5098.4 free,    464.0 used,    507.8 buff/cache
MiB Swap:   4096.0 total,   4096.0 free,      0.0 used.   5463.7 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0   24236  15084  11244 S   0.0   0.2   0:00.63 systemd
      2 root      20   0    3120   1920   1920 S   0.0   0.0   0:00.00 init-systemd(Ub
      8 root      20   0    3120   1792   1792 S   0.0   0.0   0:00.00 init
     48 root      19  -1   50404  16256  15104 S   0.0   0.3   0:00.14 systemd-journal
     83 systemd+  20   0   22400  13824  11520 S   0.0   0.2   0:00.07 systemd-resolve
     97 root      20   0   35076  11776   8960 S   0.0   0.2   0:00.17 systemd-udevd
    114 root      20   0    2880   1920   1792 S   0.0   0.0   0:00.03 chronyd-starter
    121 root      20   0    4464   2944   2816 S   0.0   0.0   0:00.00 cron
    127 message+  20   0    8816   4992   4480 S   0.0   0.1   0:00.04 dbus-daemon
    142 root      20   0   44084  29156  14208 S   0.0   0.5   0:00.17 networkd-dispat
    153 root      20   0   18404   9088   7936 S   0.0   0.1   0:00.05 systemd-logind
    156 root      20   0 1718048  14464  12416 S   0.0   0.2   0:00.07 wsl-pro-service
    175 syslog    20   0  220532   5504   4352 S   0.0   0.1   0:00.04 rsyslogd
    201 _chrony   20   0   20408   6528   5888 S   0.0   0.1   0:00.02 chronyd
    212 root      20   0    5528   2816   2688 S   0.0   0.0   0:00.00 agetty
    214 _chrony   20   0   12080   2232   1664 S   0.0   0.0   0:00.00 chronyd
    219 root      20   0  123448  32000  17536 S   0.0   0.5   0:00.13 unattended-upgr
    228 root      20   0    5484   2560   2432 S   0.0   0.0   0:00.00 agetty
    313 root      20   0    3128   1028    896 S   0.0   0.0   0:00.00 SessionLeader
    314 root      20   0    3144   1292   1152 S   0.0   0.0   0:00.00 Relay(315)
    315 Premcha+  20   0    6260   5376   3840 S   0.0   0.1   0:00.06 bash
    316 root      20   0    8636   5376   4608 S   0.0   0.1   0:00.01 login
    359 Premcha+  20   0   22332  12800  10624 S   0.0   0.2   0:00.11 systemd
    361 Premcha+  20   0   22900   4044   2048 S   0.0   0.1   0:00.00 (sd-pam)
    399 Premcha+  20   0    6128   5248   3840 S   0.0   0.1   0:00.02 bash
    547 Premcha+  20   0    8176   5760   3712 R   0.0   0.1   0:00.04 top
```

By using “TOP” we can displays running processes and the CPU usage.

To display memory usage and filesystem disk utilization

```
Premchand@Premchand:~/linux_practical$ free -h
               total        used        free      shared  buff/cache   available
Mem:           5.8Gi       465Mi       5.3Gi       3.5Mi       151Mi       5.3Gi
Swap:          4.0Gi          0B       4.0Gi
Premchand@Premchand:~/linux_practical$ df -h
Filesystem      Size  Used Avail Use% Mounted on
none            2.9G     0  2.9G   0% /usr/lib/modules/6.6.87.2-microsoft-standard-WSL2
none            2.9G  4.0K  2.9G   1% /mnt/wsl
drivers         226G  212G   14G  95% /usr/lib/wsl/drivers
/dev/sdd       1007G  1.7G  955G   1% /
none            2.9G   76K  2.9G   1% /mnt/wslg
none            2.9G     0  2.9G   0% /usr/lib/wsl/lib
rootfs          2.9G  2.7M  2.9G   1% /init
none            2.9G  496K  2.9G   1% /run
none            2.9G     0  2.9G   0% /run/lock
none            2.9G     0  2.9G   0% /run/shm
none            2.9G   76K  2.9G   1% /mnt/wslg/versions.txt
none            2.9G   76K  2.9G   1% /mnt/wslg/doc
C:\             226G  212G   14G  95% /mnt/c
D:\             100G   29G   72G  29% /mnt/d
E:\             150G   39G  112G  26% /mnt/e
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
tmpfs           2.9G     0  2.9G   0% /tmp
none            1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
none            1.0M     0  1.0M   0% /run/credentials/getty@tty1.service
none            1.0M     0  1.0M   0% /run/credentials/console-getty.service
tmpfs           593M   12K  593M   1% /run/user/1000
Premchand@Premchand:~/linux_practical$
```

Question 6: Bash Scripting

Bash script named backup_check.sh

```
Premchand@Premchand:~$ mkdir -p ~/backup_practical
Premchand@Premchand:~$ cd ~/backup_practical
Premchand@Premchand:~/backup_practical$ nano backup_check.sh
```

```
Premchand@Premchand:~/backup_practical$ chmod +x backup_check.sh
Premchand@Premchand:~/backup_practical$ nano backup_check.sh
Premchand@Premchand:~/backup_practical$ chmod +x backup_check.sh
Premchand@Premchand:~/backup_practical$ mkdir -p myfiles
Premchand@Premchand:~/backup_practical$ echo "Backup test" > myfiles/report.txt
Premchand@Premchand:~/backup_practical$ ./backup_check.sh myfiles backups
Backup successful!
Premchand@Premchand:~/backup_practical$ ls backups
backup_20261028_064359.tar.gz
```

```
Premchand@Premchand:~/backup_practical$ ./backup_check.sh
Error: Provide source and destination directories.
Usage: ./backup_check.sh source destination
Premchand@Premchand:~/backup_practical$ ./backup_check.sh wrongfolder backups
Error: Source directory does not exist.
Premchand@Premchand:~/backup_practical$
```

SECTION C: PYSPARK DATA ENGINEERING

Question 7: PySpark DataFrame Operations

Created the PySpark DataFrame and Inserted the records

```
spark = SparkSession.builder \
    .appName("EmployeeData") \
    .master("local[*]") \
    .getOrCreate()

data = [
    (1, "Prem", "IT", 75000, "2024-01-10"),
    (2, "Rahul", "HR", 45000, "2023-05-12"),
    (3, "Priya", "IT", 85000, "2022-03-15"),
    (4, "Anil", "Sales", 55000, "2024-07-20"),
    (5, "Sneha", "HR", 62000, "2021-09-11"),
    (6, "Arjun", "IT", 95000, "2020-02-18"),
    (7, "Kiran", "Sales", 70000, "2023-11-25"),
    (8, "Divya", "Finance", 65000, "2022-06-30"),
    (9, "Manoj", "IT", 58000, "2024-04-05"),
    (10, "Neha", "HR", 72000, "2020-12-01"),
    (11, "Vijay", "Finance", 80000, "2021-08-16"),
    (12, "Pooja", "Sales", 48000, "2023-02-14"),
    (13, "Ravi", "IT", 68000, "2022-10-10"),
    (14, "Meena", "Finance", 90000, "2020-07-22"),
    (15, "Ajay", "Sales", 61000, "2024-05-19")
]

columns = ["EmployeeID", "EmployeeName", "Department",
           "Salary", "JoiningDate"]

df = spark.createDataFrame(data, columns)
df = df.withColumn("JoiningDate", col("JoiningDate").cast("date"))
```

To display the schema and first five records

```
34    print("\n1. SCHEMA")
35    df.printSchema()
36
37    print("\n2. FIRST FIVE EMPLOYEES")
38    df.show(5)
```

```
1. SCHEMA
root
 |-- EmployeeID: long (nullable = true)
 |-- EmployeeName: string (nullable = true)
 |-- Department: string (nullable = true)
 |-- Salary: long (nullable = true)
 |-- JoiningDate: date (nullable = true)


2. FIRST FIVE EMPLOYEES
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|         1|        Prem|        IT| 75000| 2024-01-10|
|         2|       Rahul|        HR| 45000| 2023-05-12|
|         3|       Priya|        IT| 85000| 2022-03-15|
|         4|        Anil|     Sales| 55000| 2024-07-20|
|         5|       Sneha|        HR| 62000| 2021-09-11|
+----------+------------+----------+------+-----------+
only showing top 5 rows
```

Filter employees whose monthly salary is greater than ₹60,000

```
print("\n3. SALARY GREATER THAN 60000")
df.filter(col("Salary") > 60000).show()
```

```
3. SALARY GREATER THAN 60000
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|         1|        Prem|        IT| 75000| 2024-01-10|
|         3|       Priya|        IT| 85000| 2022-03-15|
|         5|       Sneha|        HR| 62000| 2021-09-11|
|         6|       Arjun|        IT| 95000| 2020-02-18|
|         7|       Kiran|     Sales| 70000| 2023-11-25|
|         8|       Divya|   Finance| 65000| 2022-06-30|
|        10|        Neha|        HR| 72000| 2020-12-01|
|        11|       Vijay|   Finance| 80000| 2021-08-16|
|        13|        Ravi|        IT| 68000| 2022-10-10|
|        14|       Meena|   Finance| 90000| 2020-07-22|
|        15|        Ajay|     Sales| 61000| 2024-05-19|
+----------+------------+----------+------+-----------+
```

Added an AnnualSalary column calculated from the monthly salary

```
print("\n4. ANNUAL SALARY")
df = df.withColumn("AnnualSalary", col("Salary") * 12)
df.show()
```

```
4. ANNUAL SALARY
+----------+------------+----------+------+-----------+------------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+------------+----------+------+-----------+------------+
|         1|        Prem|        IT| 75000| 2024-01-10|      900000|
|         2|       Rahul|        HR| 45000| 2023-05-12|      540000|
|         3|       Priya|        IT| 85000| 2022-03-15|     1020000|
|         4|        Anil|     Sales| 55000| 2024-07-20|      660000|
|         5|       Sneha|        HR| 62000| 2021-09-11|      744000|
|         6|       Arjun|        IT| 95000| 2020-02-18|     1140000|
|         7|       Kiran|     Sales| 70000| 2023-11-25|      840000|
|         8|       Divya|   Finance| 65000| 2022-06-30|      780000|
|         9|       Manoj|        IT| 58000| 2024-04-05|      696000|
|        10|        Neha|        HR| 72000| 2020-12-01|      864000|
|        11|       Vijay|   Finance| 80000| 2021-08-16|      960000|
|        12|       Pooja|     Sales| 48000| 2023-02-14|      576000|
|        13|        Ravi|        IT| 68000| 2022-10-10|      816000|
|        14|       Meena|   Finance| 90000| 2020-07-22|     1080000|
|        15|        Ajay|     Sales| 61000| 2024-05-19|      732000|
+----------+------------+----------+------+-----------+------------+
```

Group employees by department and calculate the average salary and employee count

```
print("\n5. DEPARTMENT-WISE AVERAGE SALARY AND COUNT")
df.groupBy("Department").agg(
    avg("Salary").alias("AverageSalary"),
    count("*").alias("EmployeeCount")
).show()
```

```
5. DEPARTMENT-WISE AVERAGE SALARY AND COUNT
+----------+------------------+-------------+
|Department|     AverageSalary|EmployeeCount|
+----------+------------------+-------------+
|        IT|           76200.0|            5|
|        HR|59666.666666666664|            3|
|     Sales|           58500.0|            4|
|   Finance| 78333.33333333333|            3|
+----------+------------------+-------------+
```

To display the five highest-paid employees

```
print("\n6. TOP FIVE HIGHEST-PAID EMPLOYEES")
df.orderBy(col("Salary").desc()).show(5)

spark.stop()
```

```
+----------+------------+----------+------+-----------+------------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+------------+----------+------+-----------+------------+
|         6|       Arjun|        IT| 95000| 2020-02-18|     1140000|
|        14|       Meena|   Finance| 90000| 2020-07-22|     1080000|
|         3|       Priya|        IT| 85000| 2022-03-15|     1020000|
|        11|       Vijay|   Finance| 80000| 2021-08-16|      960000|
|         1|        Prem|        IT| 75000| 2024-01-10|      900000|
+----------+------------+----------+------+-----------+------------+
only showing top 5 rows
```

Question 8: Data Cleaning and Aggregation

Read the CSV file using an appropriate schema and header configuration.

```
df = spark.read.csv(
    "transactions.csv",
    header=True,
    inferSchema=True
)

print("\n ORIGINAL DATA ")
df.show(truncate=False)

print("\n DATA TYPES ")
df.printSchema()
```

```
 ORIGINAL DATA
+-------------+----------+---------------+--------+--------+---------+
|TransactionID|CustomerID|TransactionDate|Product |Quantity|UnitPrice|
+-------------+----------+---------------+--------+--------+---------+
|1            |101.0     |2026-01-10     |Laptop  |2.0     |50000.0  |
|2            |102.0     |2026-01-11     |Mouse   |5.0     |500.0    |
|3            |103.0     |2026-01-12     |Keyboard|3.0     |1500.0   |
|1            |101.0     |2026-01-10     |Laptop  |2.0     |50000.0  |
|5            |104.0     |invalid-date   |Monitor |1.0     |12000.0  |
|6            |105.0     |2026-01-15     |Mouse   |4.0     |500.0    |
|7            |106.0     |2026-01-16     |Laptop  |1.0     |50000.0  |
|8            |107.0     |2026-01-17     |Keyboard|2.0     |1500.0   |
|9            |NULL      |2026-01-18     |Monitor |2.0     |12000.0  |
|10           |109.0     |2026-01-19     |NULL    |1.0     |500.0    |
|11           |110.0     |2026-02-01     |Laptop  |1.0     |50000.0  |
|12           |111.0     |2026-02-03     |Mouse   |NULL    |500.0    |
|13           |112.0     |2026-02-05     |Monitor |2.0     |12000.0  |
|14           |113.0     |05/02/2026     |Keyboard|4.0     |1500.0   |
|15           |114.0     |2026-02-07     |Mouse   |3.0     |500.0    |
|16           |115.0     |2026-02-08     |Monitor |1.0     |NULL     |
+-------------+----------+---------------+--------+--------+---------+


 DATA TYPES
root
 |-- TransactionID: integer (nullable = true)
 |-- CustomerID: double (nullable = true)
 |-- TransactionDate: string (nullable = true)
 |-- Product: string (nullable = true)
 |-- Quantity: double (nullable = true)
 |-- UnitPrice: double (nullable = true)
```

To identify and remove duplicate transactions

and Handle missing values in required fields

and Convert TransactionDate into a date type and handle invalid dates

```
df = df.dropDuplicates(["TransactionID"])

df = df.dropna(
    subset=[
        "TransactionID",
        "CustomerID",
        "TransactionDate",
        "Product",
        "Quantity",
        "UnitPrice"
    ]
)

df = df.withColumn(
    "TransactionDate",
    to_date(
        try_to_timestamp(
            col("TransactionDate"),
            lit("yyyy-MM-dd")
        )
    )
)
```

CLEANED DATA

TransactionID	CustomerID	TransactionDate	Product	Quantity	UnitPrice	Revenue
1	101.0	2026-01-10	Laptop	2.0	50000.0	100000.0
2	102.0	2026-01-11	Mouse	5.0	500.0	2500.0
3	103.0	2026-01-12	Keyboard	3.0	1500.0	4500.0
6	105.0	2026-01-15	Mouse	4.0	500.0	2000.0
7	106.0	2026-01-16	Laptop	1.0	50000.0	50000.0
8	107.0	2026-01-17	Keyboard	2.0	1500.0	3000.0
11	110.0	2026-02-01	Laptop	1.0	50000.0	50000.0
13	112.0	2026-02-05	Monitor	2.0	12000.0	24000.0
15	114.0	2026-02-07	Mouse	3.0	500.0	1500.0

Calculate revenue using Quantity × UnitPrice.

And Calculate total revenue by product and display the results in descending order

```
print("\n TOTAL REVENUE BY PRODUCT ")

revenue_df = df.groupBy("Product").agg(
    spark_sum("Revenue").alias("TotalRevenue")
)

revenue_df = revenue_df.orderBy(
    col("TotalRevenue").desc()
)

revenue_df.show(truncate=False)

print("\n TOTAL REVENUE ")

df.select(
    spark_sum("Revenue").alias("TotalRevenue")
).show()

print("\n TOTAL VALID TRANSACTIONS ")
print("Total Transactions:", df.count())
```

```
 TOTAL REVENUE BY PRODUCT
+--------+------------+
|Product |TotalRevenue|


 TOTAL REVENUE BY PRODUCT
+--------+------------+
|Product |TotalRevenue|
 TOTAL REVENUE BY PRODUCT
+--------+------------+
|Product |TotalRevenue|
+--------+------------+
|Laptop  |200000.0    |
|Monitor |24000.0     |
|Keyboard|7500.0      |
|Mouse   |6000.0      |
+--------+------------+
|Mouse   |6000.0      |
+--------+------------+
+--------+------------+


 TOTAL REVENUE
+------------+
|TotalRevenue|
+------------+
|    237500.0|
+------------+


 TOTAL VALID TRANSACTIONS
Total Transactions: 9
```

Question 9: PySpark Joins and Performance

Loaded both datasets into DataFrames

```
customers_df = spark.read.csv(
    "customers.csv",
    header=True,
    inferSchema=True
)

orders_df = spark.read.csv(
    "orders.csv",
    header=True,
    inferSchema=True
)


print("\n CUSTOMERS DATA")
customers_df.show(truncate=False)


print("\n ORDERS DATA ")
orders_df.show(truncate=False)
```

```
 CUSTOMERS DATA
+----------+------------+---------+
|CustomerID|CustomerName|City     |
+----------+------------+---------+
|101       |Rahul       |Bengaluru|
|102       |Priya       |Hyderabad|
|103       |Amit        |Chennai  |
|104       |Sneha       |Mumbai   |
|105       |Kiran       |Bengaluru|
+----------+------------+---------+


 ORDERS DATA
+-------+----------+-----------+----------+
|OrderID|CustomerID|OrderAmount|OrderDate |
+-------+----------+-----------+----------+
|1001   |101       |2500       |2026-10-01|
|1002   |102       |1500       |2026-10-02|
|1003   |101       |3000       |2026-10-03|
|1004   |103       |800        |2026-10-04|
|1005   |105       |2200       |2026-10-05|
+-------+----------+-----------+----------+
```

Join the datasets to display orders with customer details

```
print("\n ORDERS WITH CUSTOMER DETAILS ")

joined_df = customers_df.join(
    orders_df,
    on="CustomerID",
    how="inner"
)

joined_df.show(truncate=False)
```

```
 ORDERS WITH CUSTOMER DETAILS
+----------+------------+---------+-------+-----------+----------+
|CustomerID|CustomerName|City     |OrderID|OrderAmount|OrderDate |
+----------+------------+---------+-------+-----------+----------+
|101       |Rahul       |Bengaluru|1001   |2500       |2026-10-01|
|102       |Priya       |Hyderabad|1002   |1500       |2026-10-02|
|101       |Rahul       |Bengaluru|1003   |3000       |2026-10-03|
|103       |Amit        |Chennai  |1004   |800        |2026-10-04|
|105       |Kiran       |Bengaluru|1005   |2200       |2026-10-05|
+----------+------------+---------+-------+-----------+----------+
```

Identify customers who have never placed an order.

```
print("\n CUSTOMERS WITHOUT ORDERS ")

customers_without_orders = customers_df.join(
    orders_df,
    on="CustomerID",
    how="left_anti"
)

customers_without_orders.show(truncate=False)
```

```
  CUSTOMERS WITHOUT ORDERS
 +----------+------------+------+
 |CustomerID|CustomerName|City  |
 +----------+------------+------+
 |104       |Sneha       |Mumbai|
 +----------+------------+------+
```

Calculate the total order value by city

```
print("\n TOTAL ORDER VALUE BY CITY ")

city_revenue_df = joined_df.groupBy("City").agg(
    spark_sum("OrderAmount").alias("TotalOrderValue")
)

city_revenue_df = city_revenue_df.orderBy(
    col("TotalOrderValue").desc()
)

city_revenue_df.show(truncate=False)
```

```
 TOTAL ORDER VALUE BY CITY
+---------+---------------+
|City     |TotalOrderValue|
+---------+---------------+
|Bengaluru|7700           |
|Hyderabad|1500           |
|Chennai  |800            |
+---------+---------------+
```

broadcast join is useful and what conditions make it appropriate

A broadcast join is useful when one DataFrame is small enough to fit in the memory of each worker node. Spark sends a copy of the smaller DataFrame to the workers, which can avoid shuffling the larger DataFrame and improve join performance.

```
print("\n BROADCAST JOIN EXAMPLE ")

broadcast_join_df = customers_df.join(
    broadcast(orders_df),
    on="CustomerID",
    how="inner"
)

broadcast_join_df.show(truncate=False)
```

```
 BROADCAST JOIN EXAMPLE
+----------+------------+---------+-------+-----------+----------+
|CustomerID|CustomerName|City     |OrderID|OrderAmount|OrderDate |
+----------+------------+---------+-------+-----------+----------+
|101       |Rahul       |Bengaluru|1003   |3000       |2026-10-03|
|101       |Rahul       |Bengaluru|1001   |2500       |2026-10-01|
|102       |Priya       |Hyderabad|1002   |1500       |2026-10-02|
|103       |Amit        |Chennai  |1004   |800        |2026-10-04|
|105       |Kiran       |Bengaluru|1005   |2200       |2026-10-05|
+----------+------------+---------+-------+-----------+----------+
```

the difference between transformations and actions

Transformation: Creates a new DataFrame or describes a computation.

Examples: filter(), select(), join(), groupBy().

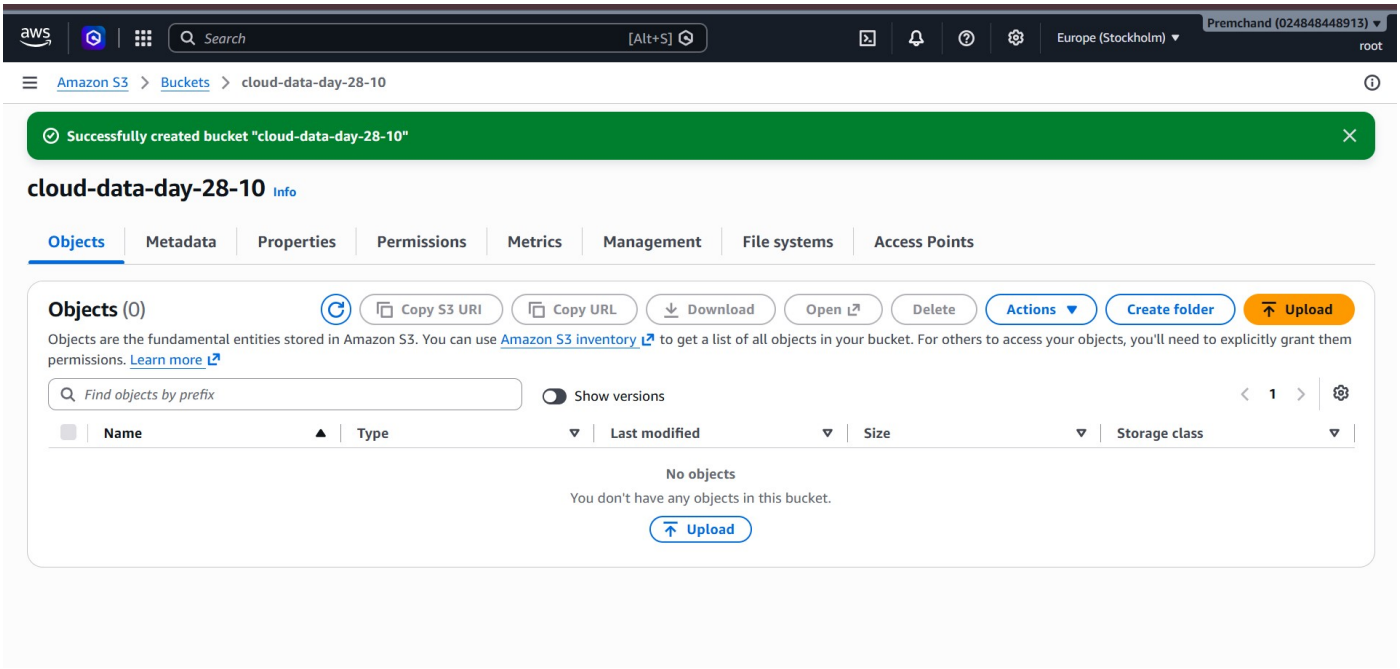
Action: Triggers execution and returns or displays a result.

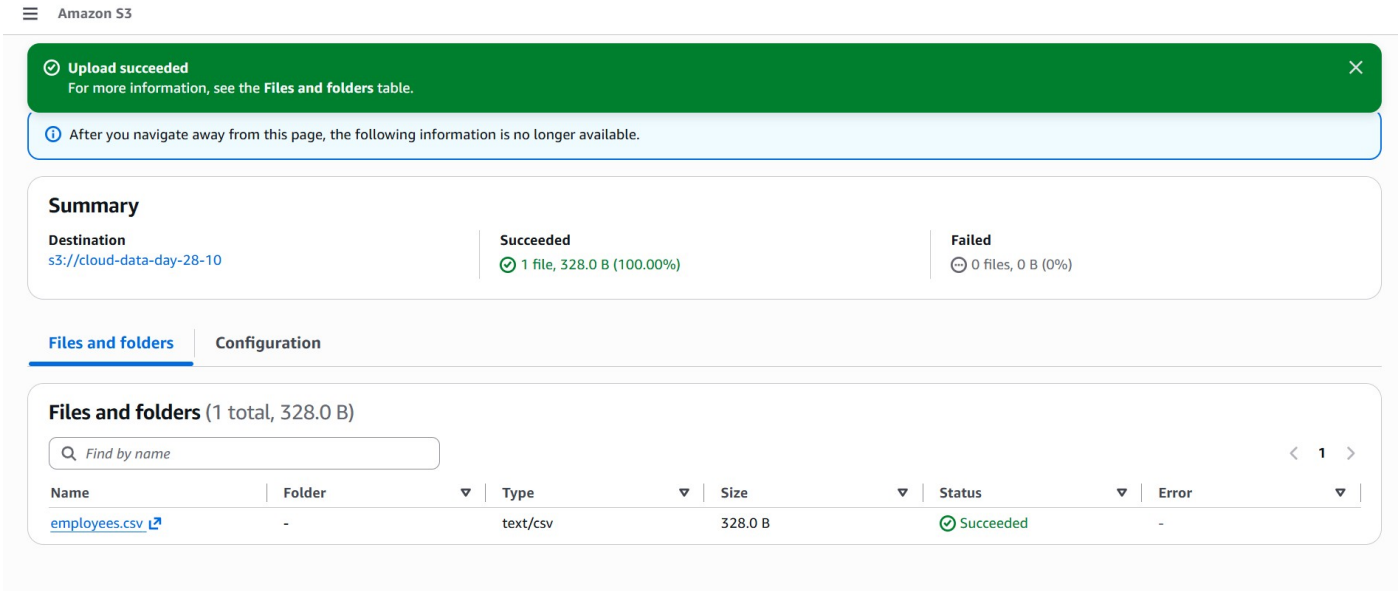
Examples: show(), count(), collect().

SECTION D: AWS CLOUD SERVICES

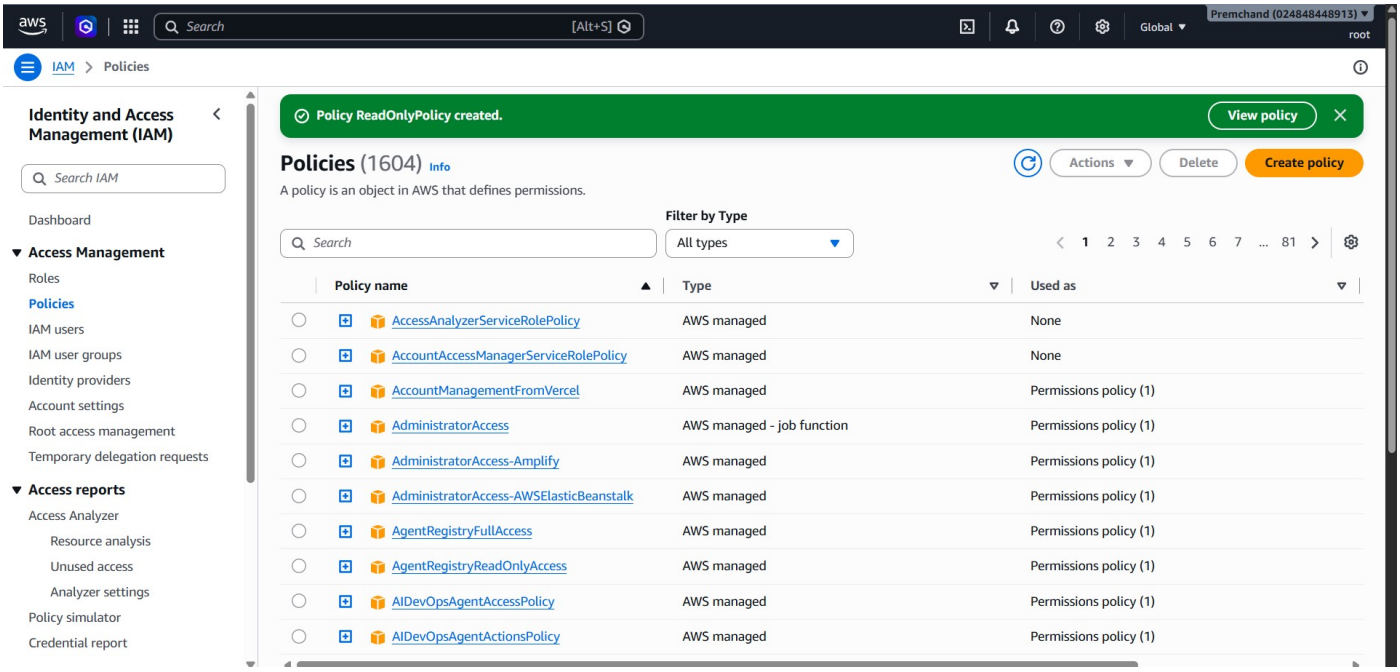
Question 10: S3 and IAM Security

Created an S3 bucket with Block Public Access enabled and uploaded a sample file





Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets.



how you would verify that an unauthorized user cannot read the file

I would log in using an IAM user who does not have permission to access the S3 bucket and try to open or download the file. AWS should show an Access Denied message. Then, I would check that an authorized HR user can access the file successfully.

how encryption at rest protects the stored data

Encryption at rest protects files stored in Amazon S3 by converting the data into an unreadable format. Only authorized access can decrypt and read the data. This helps keep confidential employee reports safe even if someone gets access to the stored data without the required permissions.

Question 11: EC2 Troubleshooting and CloudWatch

Check the EC2 instance state and status checks

I would open the AWS Console and go to EC2 → Instances. I would check whether the instance is running and whether the system and instance status checks have passed. If any check fails, I would investigate the problem and fix it.

How security groups and route tables affect connectivity

Security groups act like firewalls and control incoming and outgoing traffic. If the required port, such as 80 for HTTP or 443 for HTTPS, is blocked, users may not be able to access the application. Route tables control where network traffic goes. If the routes are incorrect or missing, the application may not be reachable over the internet or through the required network.

Configure a CloudWatch alarm when CPU utilization exceeds 85%
I would follow these steps:
1. Open AWS CloudWatch -> Alarms -> Create alarm.
2. Select the EC2 instance's CPUUtilization metric.
3. Choose Average as the statistic and set the period to 5 minutes.
4. Set the alarm condition to CPU utilization greater than 85%.

5. Set the evaluation period, for example, 3 consecutive periods of 5 minutes each.
6. Configure an Amazon SNS notification to alert the administrator.
7. Give the alarm a name, such as High-CPU-Alarm, and create it.

Question 12: Event-Driven Processing

Service for processing code

I would use AWS Lambda to run the code that checks and processes CSV files automatically without managing a server.

How an S3 upload triggers processing

When a CSV file is uploaded to an S3 bucket, an S3 event automatically triggers the Lambda function to process the file.

IAM permissions required

The Lambda function needs permission to:

Read files from the input S3 bucket using s3:GetObject

Write processed results to the output S3 bucket using s3:PutObject

These permissions should be given only for the required buckets.

How to capture errors and logs

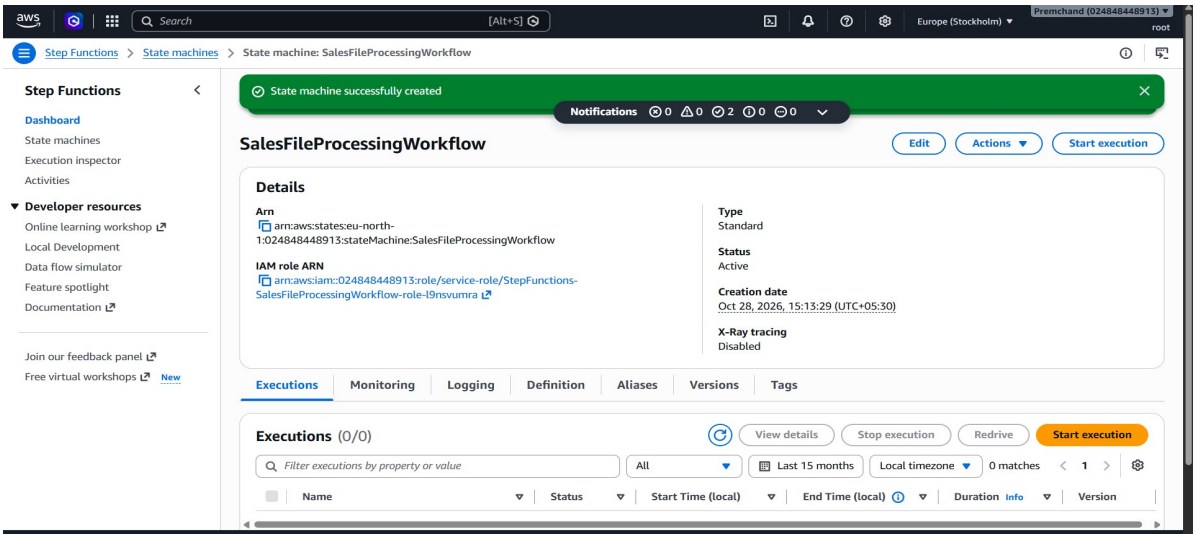
I would use Amazon CloudWatch Logs to record the processing details and errors. This helps me find and fix problems. I can also set up email notifications for failures.

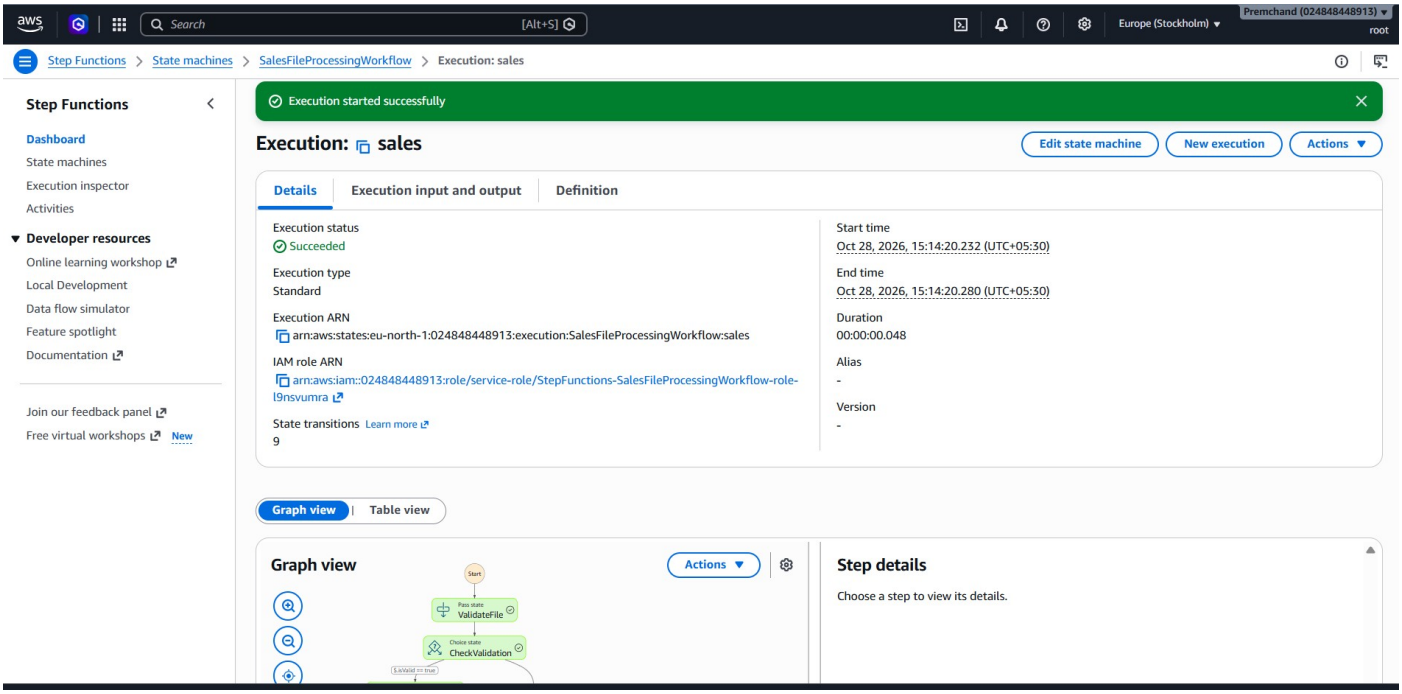
How to prevent duplicate processing or repeated triggering

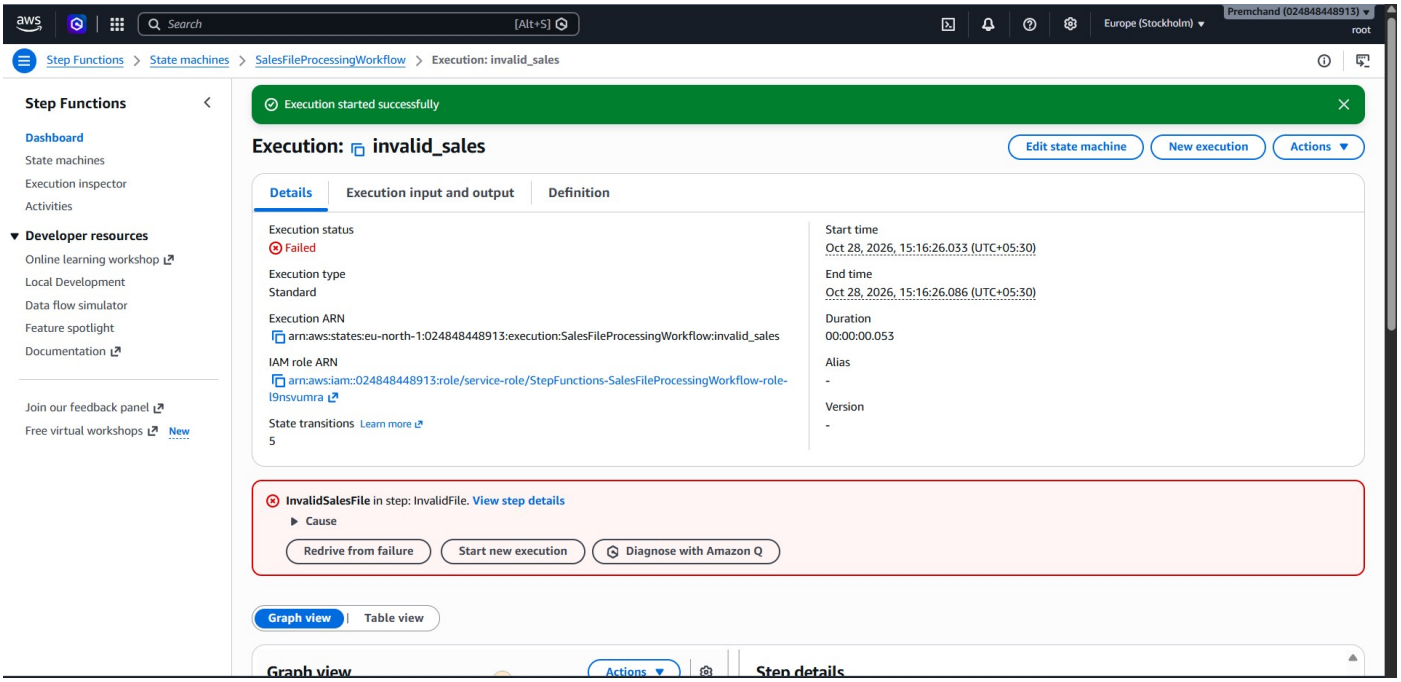
I would track processed files to avoid processing the same file multiple times. I would also save the results in a separate output bucket so that uploading the results does not trigger the Lambda function again.

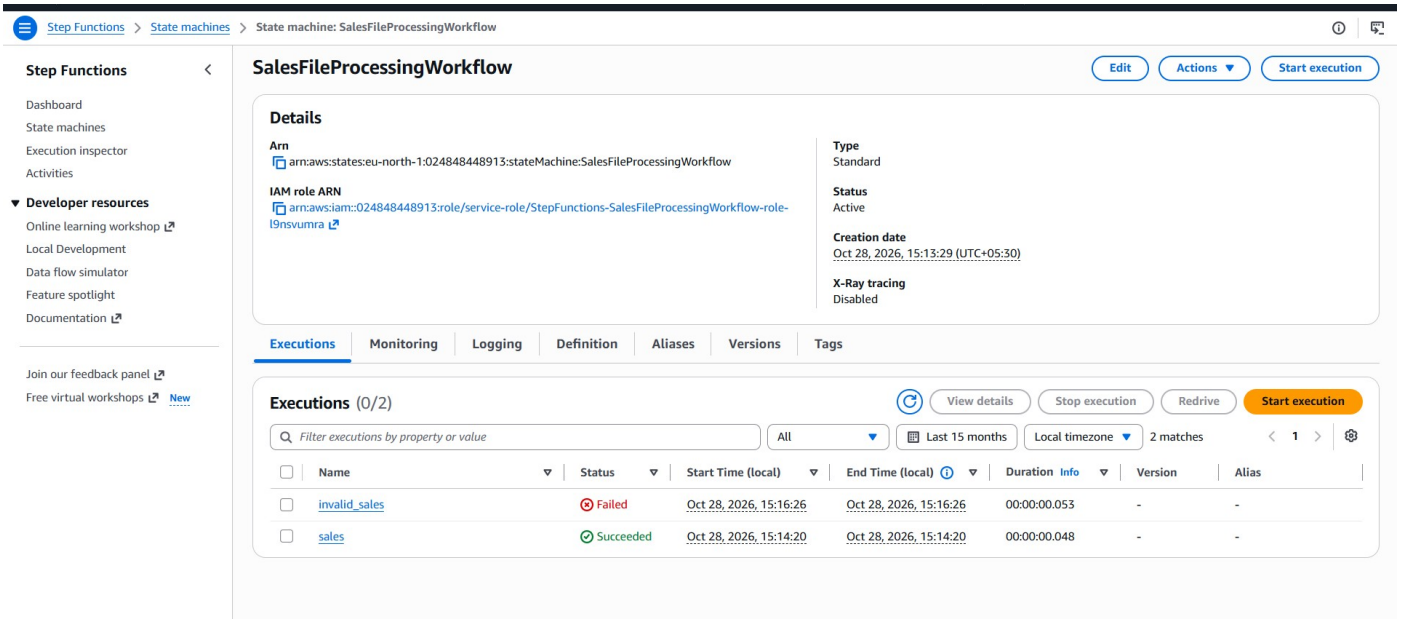
SECTION E: AWS STEP FUNCTIONS

Question 13: State Machine Design




Step Functions > State machines > SalesFileProcessingWorkflow > Execution: sales
Execution started successfully
Execution: sales
Edit state machine
New execution
Actions
Details
Execution input and output
Definition
Execution status
Succeeded
Execution type
Standard
Execution ARN
arn:aws:states:eu-north-1:024848448913:execution:SalesFileProcessingWorkflow:sales
IAM role ARN
arn:aws:iam::024848448913:role/service-role/StepFunctions-SalesFileProcessingWorkflow-role-l9nsvumra
State transitions
9
Start time
Oct 28, 2026, 15:14:20.232 (UTC+05:30)
End time
Oct 28, 2026, 15:14:20.280 (UTC+05:30)
Duration
00:00:00.048
Alias
-
Version
-
Graph view
Table view
Graph view
Actions
Step details
Choose a step to view its details.


Step Functions > State machines > SalesFileProcessingWorkflow > Execution: invalid_sales
Execution started successfully
Execution: invalid_sales
Edit state machine
New execution
Actions
Details
Execution input and output
Definition
Execution status
Failed
Execution type
Standard
Execution ARN
arn:aws:states:eu-north-1:024848448913:execution:SalesFileProcessingWorkflow:invalid_sales
IAM role ARN
arn:aws:iam::024848448913:role/service-role/StepFunctions-SalesFileProcessingWorkflow-role-l9nsvumra
State transitions
5
Start time
Oct 28, 2026, 15:16:26.033 (UTC+05:30)
End time
Oct 28, 2026, 15:16:26.086 (UTC+05:30)
Duration
00:00:00.053
Alias
-
Version
-
InvalidSalesFile in step: InvalidFile. View step details
Cause
Redrive from failure
Start new execution
Diagnose with Amazon Q
Graph view
Table view


Step Functions > State machines > State machine: SalesFileProcessingWorkflow
SalesFileProcessingWorkflow
Edit
Actions
Start execution
Details
Arn
arn:aws:states:eu-north-1:024848448913:stateMachine:SalesFileProcessingWorkflow
IAM role ARN
arn:aws:iam::024848448913:role/service-role/StepFunctions-SalesFileProcessingWorkflow-role-l9nsvumra
Type
Standard
Status
Active
Creation date
Oct 28, 2026, 15:13:29 (UTC+05:30)
X-Ray tracing
Disabled
Executions
Monitoring
Logging
Definition
Aliases
Versions
Tags
Executions (0/2)
View details
Stop execution
Redrive
Start execution
Filter executions by property or value
All
Last 15 months
Local timezone
2 matches
Name
Status
Start Time (local)
End Time (local)
Duration
Version
Alias
invalid_sales
Failed
Oct 28, 2026, 15:16:26
Oct 28, 2026, 15:16:26
00:00:00.053
sales
Succeeded
Oct 28, 2026, 15:14:20
Oct 28, 2026, 15:14:20
00:00:00.048

Identify suitable states for validation, processing, storage, and notification

The workflow uses Validate File, Transform Data, Calculate Sales, Store Results in S3, and Send Notification states to process the sales file step by step.

Explain how a Choice state can route invalid files to a failure path and valid files to processing

The Choice state checks whether the input file is valid. If valid, processing continues. If invalid, the workflow stops with a failure state.

Explain how Retry and Catch can handle transient failures and permanent errors

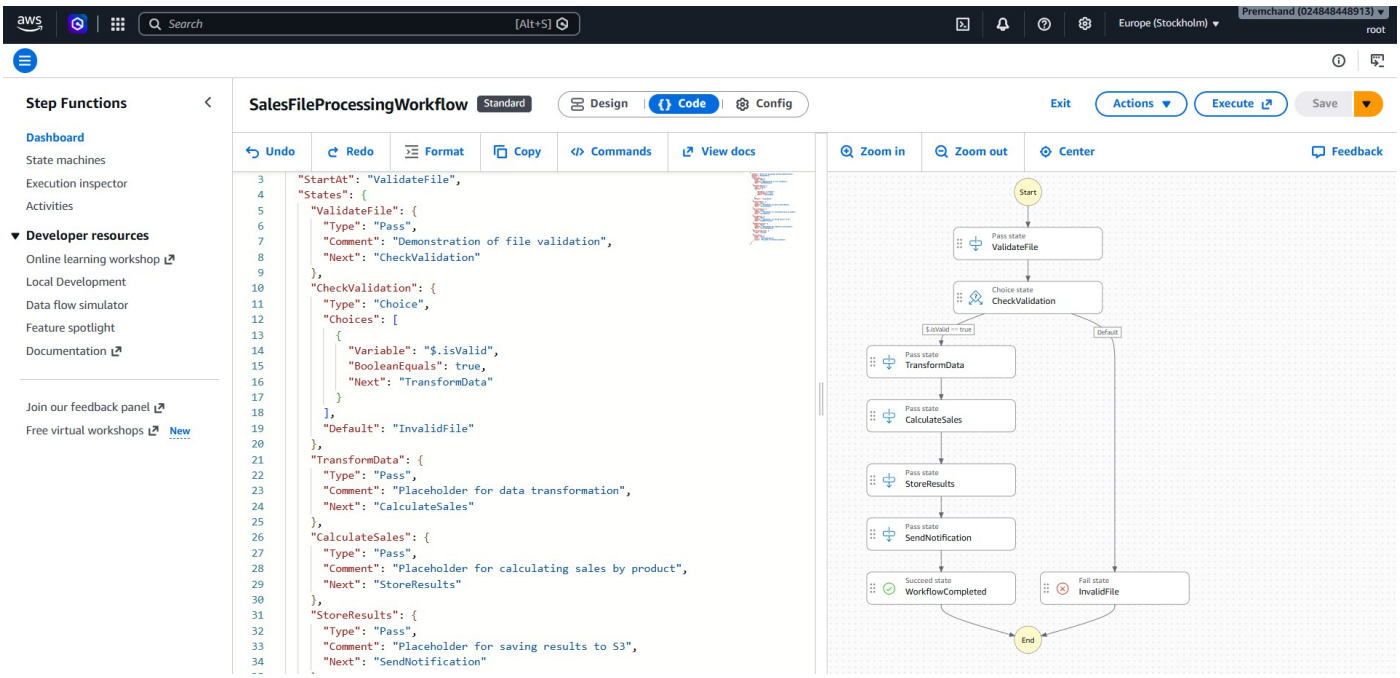
Retry tries an operation again when temporary errors occur. Catch handles errors that cannot be resolved and directs the workflow to a failure state.

Explain how the state machine can invoke Lambda functions or other supported AWS services

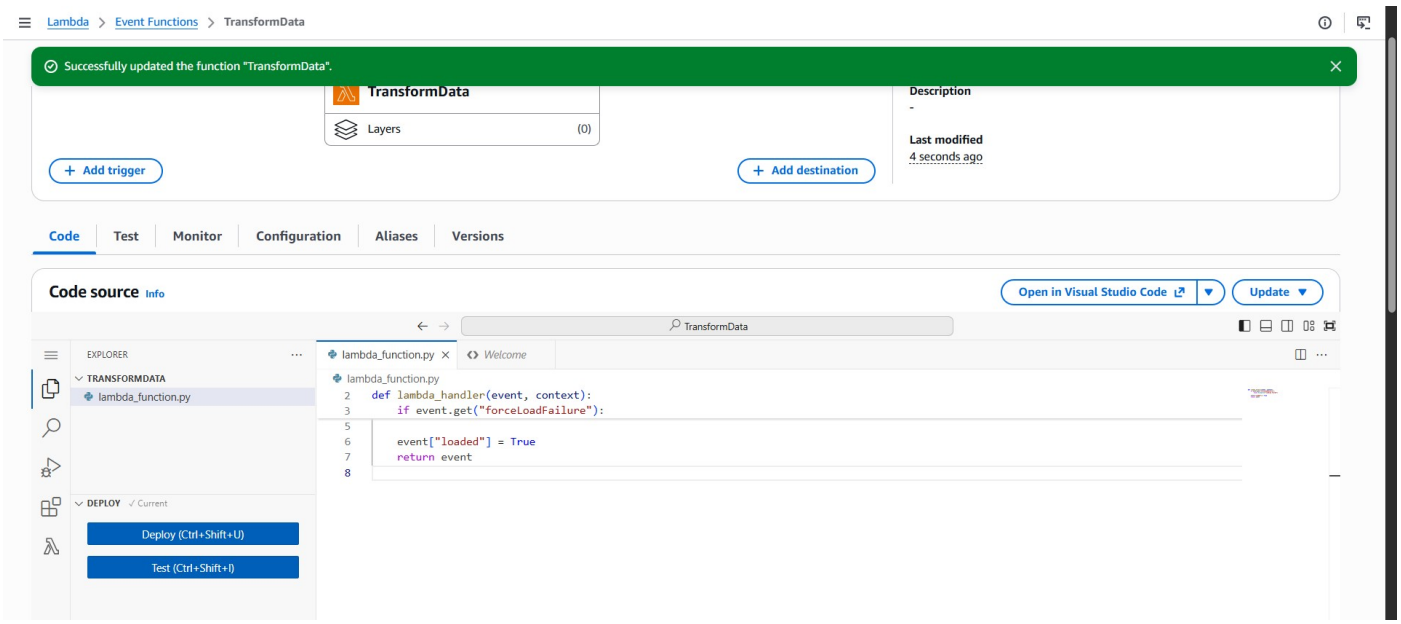
AWS Lambda performs validation and data processing, Amazon S3 stores the results, and Amazon SNS sends completion notifications.

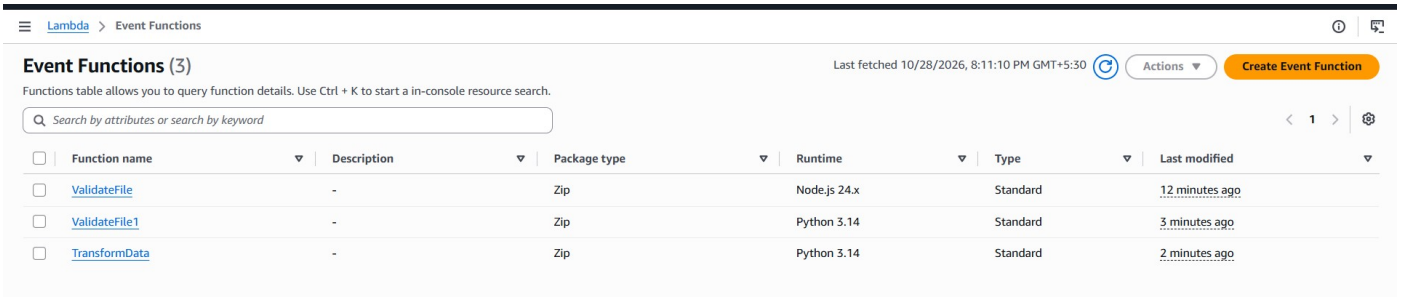
Explain how execution history helps troubleshoot a failed workflow

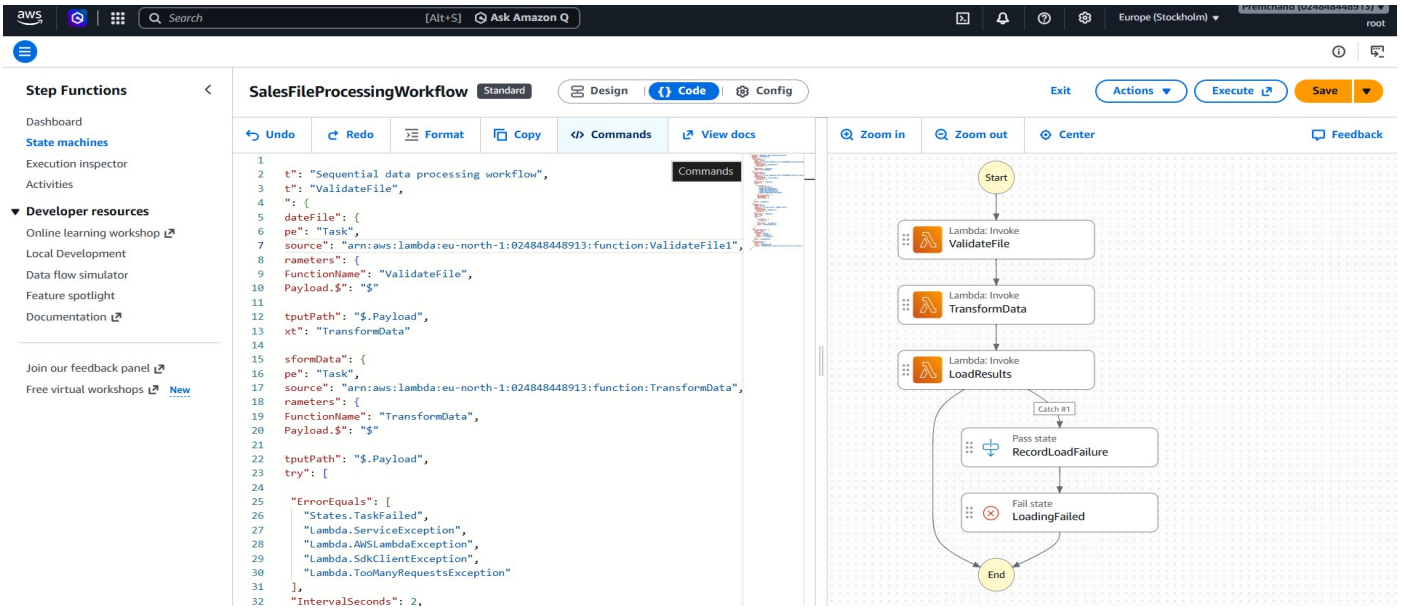
Execution history shows each steps status, input, output, and error details. It helps administrators identify and fix workflow failures.

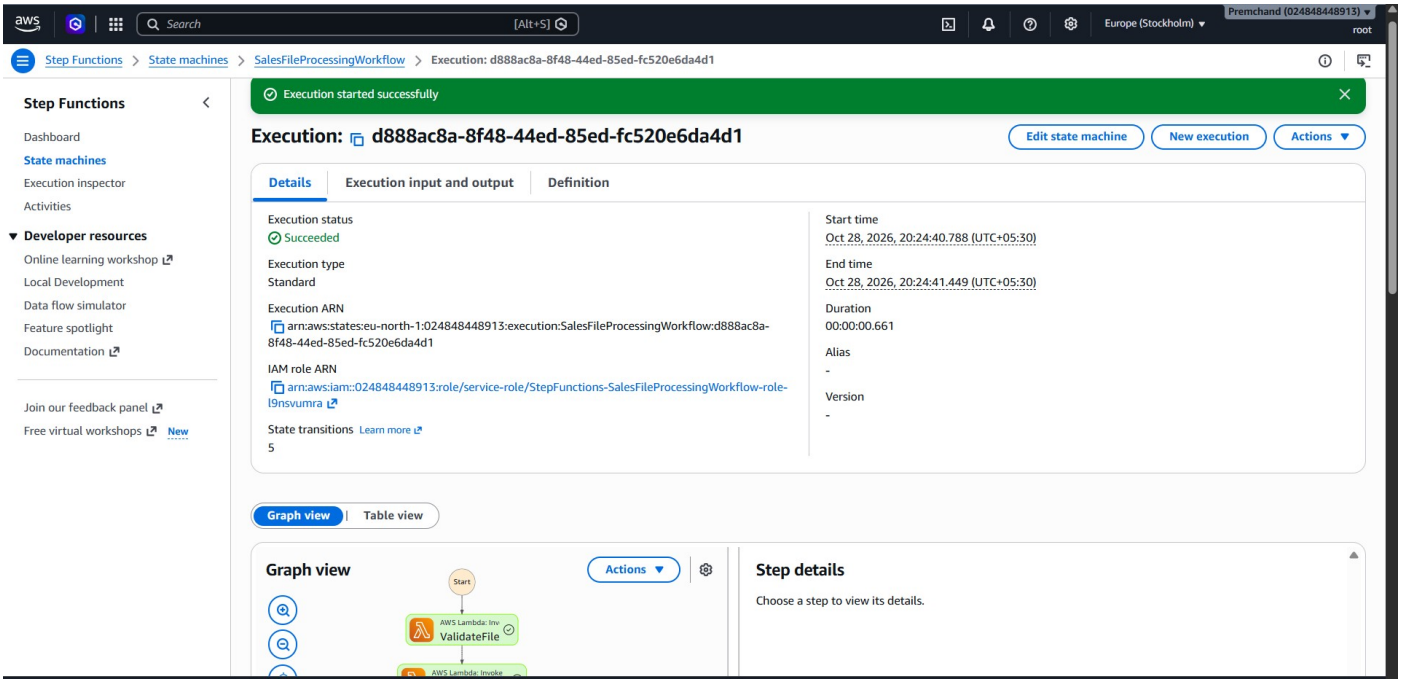


Question 14: Step Functions Implementation









a. ASL JSON Definition
The workflow uses three tasks: ValidateFile, TransformData, and LoadResults. They execute one after another to process the data.

b.Retry Policy
If TransformData fails due to a temporary error, the workflow tries again up to 3 times, starting with a 2-second wait. This helps recover from temporary problems.

c. Catch Transition
If LoadResults fails, the Catch block sends the workflow to RecordFailure, which records the error.

d. Data Passing
The output of each task is passed as input to the next task, allowing the data to move through the workflow.

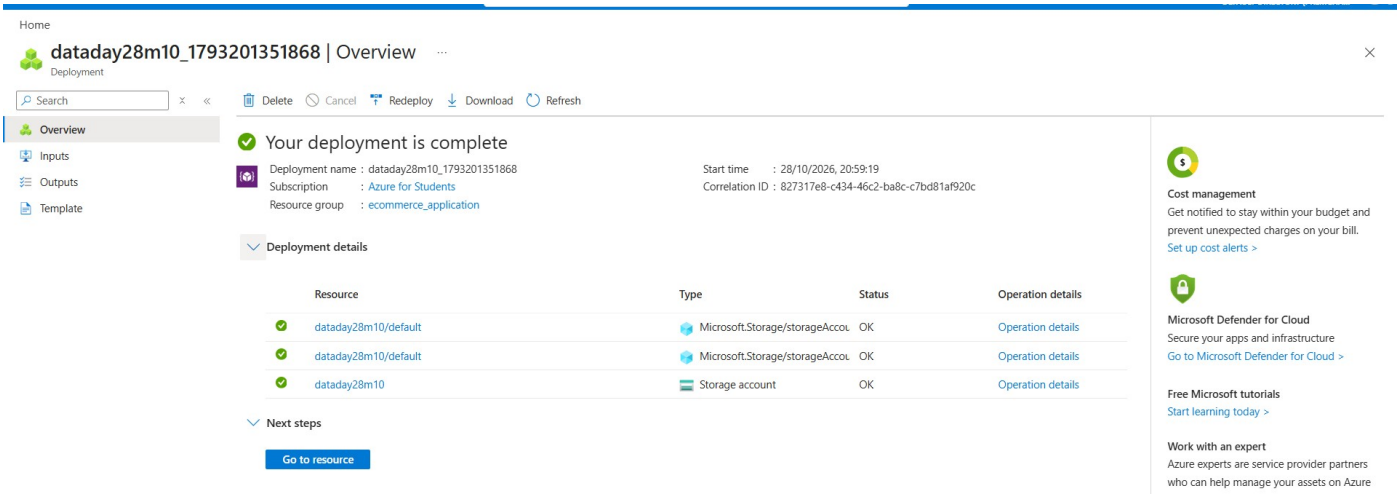
e. Testing
Test the workflow with valid input and check that all tasks succeed. Then test failures in validation, transformation, and loading. Check the Step Functions execution history to see errors, retries, and task status.

SECTION F: MICROSOFT AZURE

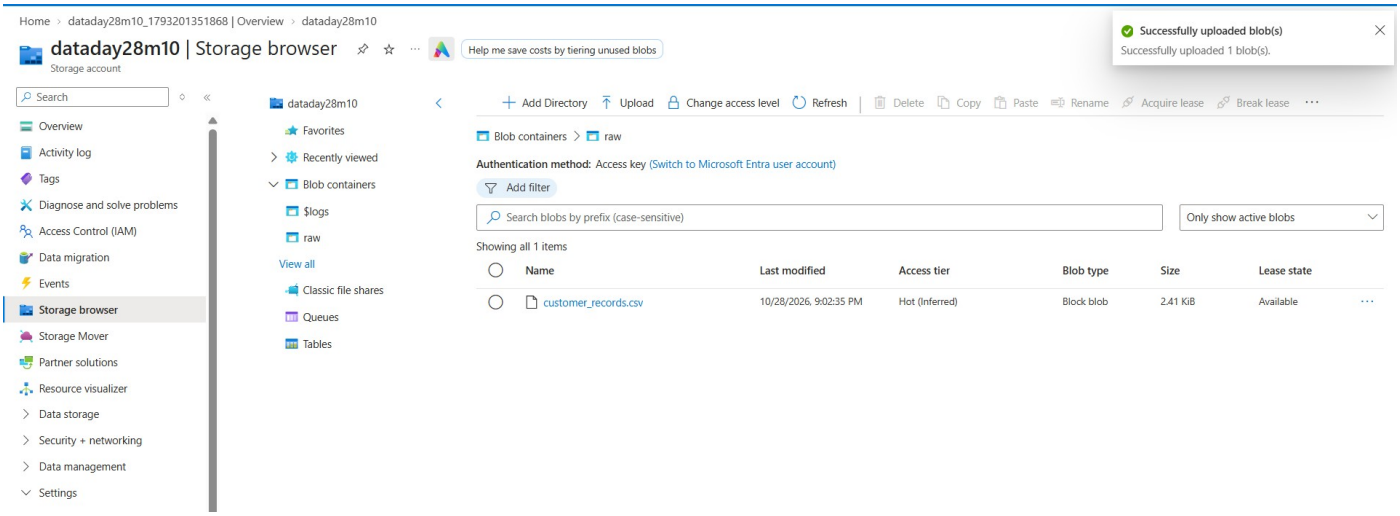
Question 15: Azure Data Lake Storage and Security

Configure or describe a storage account with hierarchical namespace enabled for Azure Data Lake Storage Gen2

Created a Azure Storage Account and enable Hierarchical Namespace to use Azure Data Lake Storage Gen2



Create a filesystem or container and upload a sample dataset



how Microsoft Entra ID and Azure role-based access control can manage access to the data

Microsoft Entra ID manages user login and identity. Azure RBAC gives permissions based on user roles. For example, an analyst can read data, while a data engineer can upload and modify files

Explain how you would verify that unauthorized users cannot access the dataset.

Log in with an unauthorized user and try to open the file. If access is denied, the security settings are working correctly.

Question 16: Azure VM Troubleshooting

Verify the VM's provisioning and power states.

Open the Azure Portal and check the VM's provisioning state is Succeeded and power state is Running.

Explain how Network Security Group rules can prevent connectivity.

NSG rules control network traffic. If the required port, such as SSH (22) or RDP (3389), is blocked, administrators cannot connect.

Explain how subnet and routing configuration affect connectivity.

Incorrect subnet settings or route tables can prevent network traffic from reaching the VM. Check the virtual network and routing configuration.

Recommend a secure troubleshooting method that avoids unnecessarily exposing the VM to the public internet.

Use Azure Bastion to connect to the VM securely through the Azure Portal. Check network settings and allow only trusted connections. Avoid exposing SSH or RDP ports to the entire public internet.

Question 17: Azure Data Integration

Select an appropriate service to orchestrate data movement.

We can use Azure Data Factory (ADF) to move data from one location to another and manage the data processing steps automatically.

Explain how to copy the CSV file into Azure Data Lake Storage Gen2.

Create a pipeline in Azure Data Factory and use the Copy Data activity to copy the customer transaction CSV file into Azure Data Lake Storage Gen2.

Describe how to organize files by ingestion date.

Store files in separate folders based on the date they are received.

example, create folders for year, month, and day. this makes it easy to find and analyze daily transaction data.

Explain how to handle malformed records and missing values.

Check the CSV file for missing values, incorrect formats, and duplicate records. Correct the data wherever possible and move invalid records into a separate error folder for checking.

Explain how to monitor pipeline execution and validate data completeness.

Use the Monitor section in Azure Data Factory to check whether the pipeline completed successfully. Compare the number of records in the original file with the copied file to make sure all data was transferred correctly.